

Week 3: importing | plotting

**NRSC 7657 Workshop in Advanced Programming for
Neuroscientists**

Built - ins

Python » English » 3.9.5 » Documentation » The Python Standard Library »

Quick search

Previous topic

Introduction

Next topic

Built-in Constants

This Page

Report a Bug

Show Source

Built-in Functions ¶

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

Built-in Functions				
<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

Imports

- Why? Because you want to use something besides built-ins

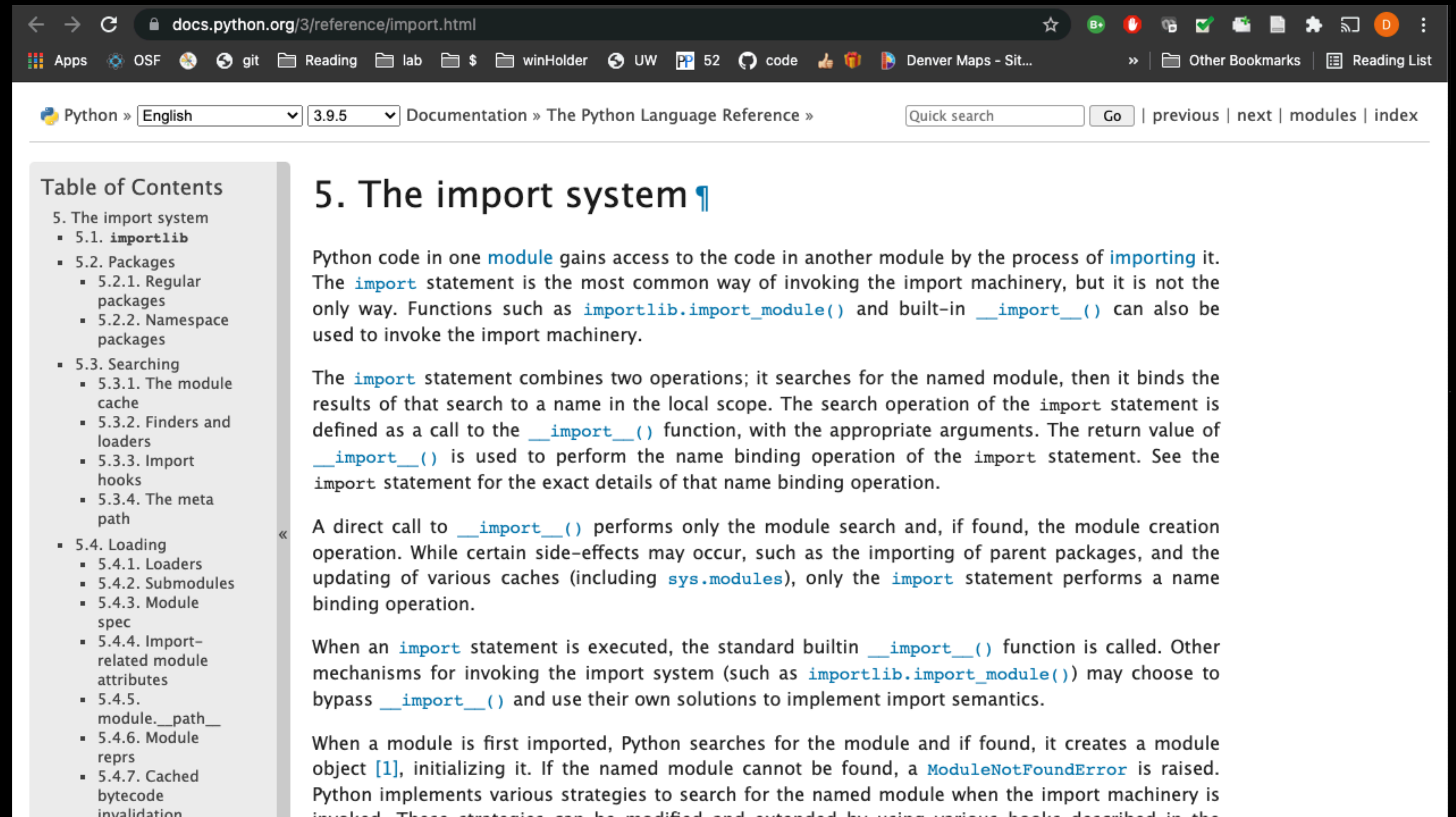
```
[ ]: import pynwb|
```

- You've got functions you wrote in a notebook, like last week. Now you have a new notebook. How do you get your (cool, useful) functions to your new notebook? Or a script? Or over to your labmates's notebook?

```
[ ]: from dlab import sorting_quality as sq|
```

Imports Modules

- **Find** a module
- **Give** it a name in the namespace



The screenshot shows the Python documentation page for the import system. The browser address bar displays `docs.python.org/3/reference/import.html`. The page header includes navigation links for Python, English, 3.9.5, and Documentation, along with a search bar and links for previous, next, modules, and index. The left sidebar contains a Table of Contents for section 5, 'The import system', with sub-items like 5.1. `importlib`, 5.2. Packages, 5.3. Searching, 5.4. Loading, and 5.5. Module attributes. The main content area is titled '5. The import system' and contains three paragraphs explaining the import process, the `import` statement, and the `__import__()` function.

Python » English » 3.9.5 » Documentation » The Python Language Reference » Quick search Go | previous | next | modules | index

5. The import system ¶

Python code in one `module` gains access to the code in another module by the process of `importing` it. The `import` statement is the most common way of invoking the import machinery, but it is not the only way. Functions such as `importlib.import_module()` and built-in `__import__()` can also be used to invoke the import machinery.

The `import` statement combines two operations; it searches for the named module, then it binds the results of that search to a name in the local scope. The search operation of the `import` statement is defined as a call to the `__import__()` function, with the appropriate arguments. The return value of `__import__()` is used to perform the name binding operation of the `import` statement. See the `import` statement for the exact details of that name binding operation.

A direct call to `__import__()` performs only the module search and, if found, the module creation operation. While certain side-effects may occur, such as the importing of parent packages, and the updating of various caches (including `sys.modules`), only the `import` statement performs a name binding operation.

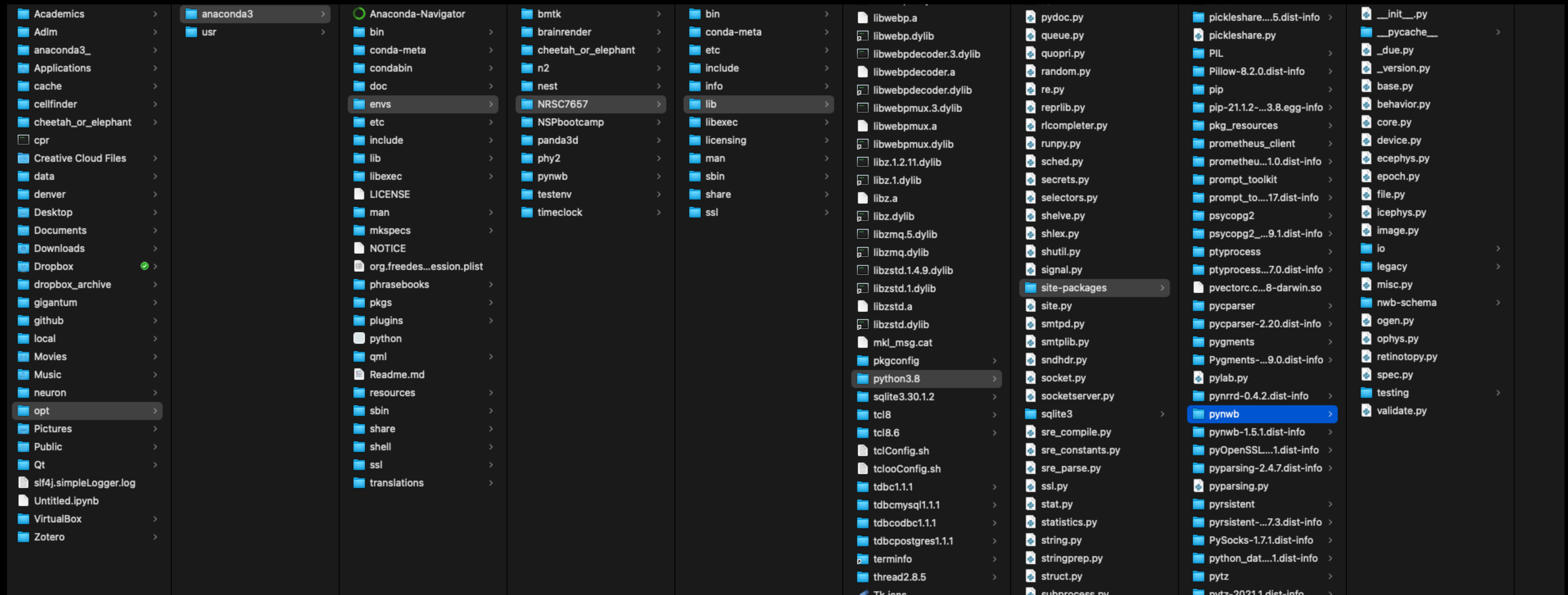
When an `import` statement is executed, the standard builtin `__import__()` function is called. Other mechanisms for invoking the import system (such as `importlib.import_module()`) may choose to bypass `__import__()` and use their own solutions to implement import semantics.

When a module is first imported, Python searches for the module and if found, it creates a module object [1], initializing it. If the named module cannot be found, a `ModuleNotFoundError` is raised. Python implements various strategies to search for the named module when the import machinery is invoked. These strategies can be modified and extended by using various hooks described in the

Imports

Finding: Absolute and relative

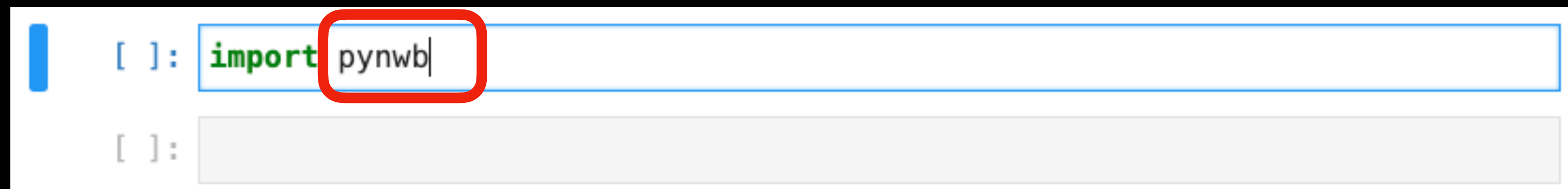
- Import has to know where to look



Imports

Naming

- Import has to know where's to map the module it found to in the namespace

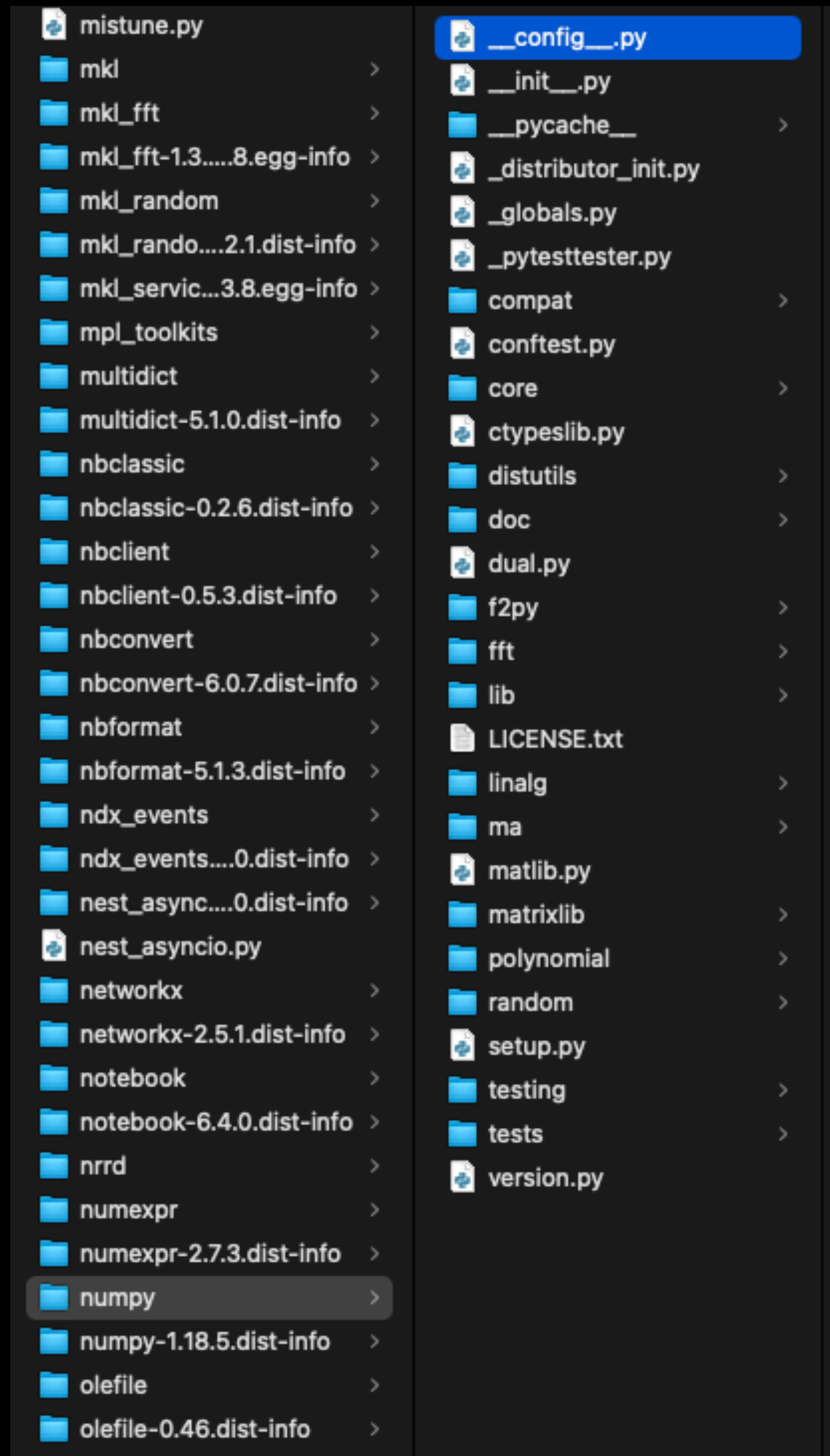


A screenshot of a code editor with a dark background. The first line of code is `[ ]: import pynwb|`. The word `import` is in green, and `pynwb|` is in white. A red rounded rectangle highlights the `pynwb|` part. Below this line is a second line `[ ]:` followed by a light gray rectangular box, indicating a new line of code is being entered.

Imports

Module structure

- Folder, init



Imports

Create a package

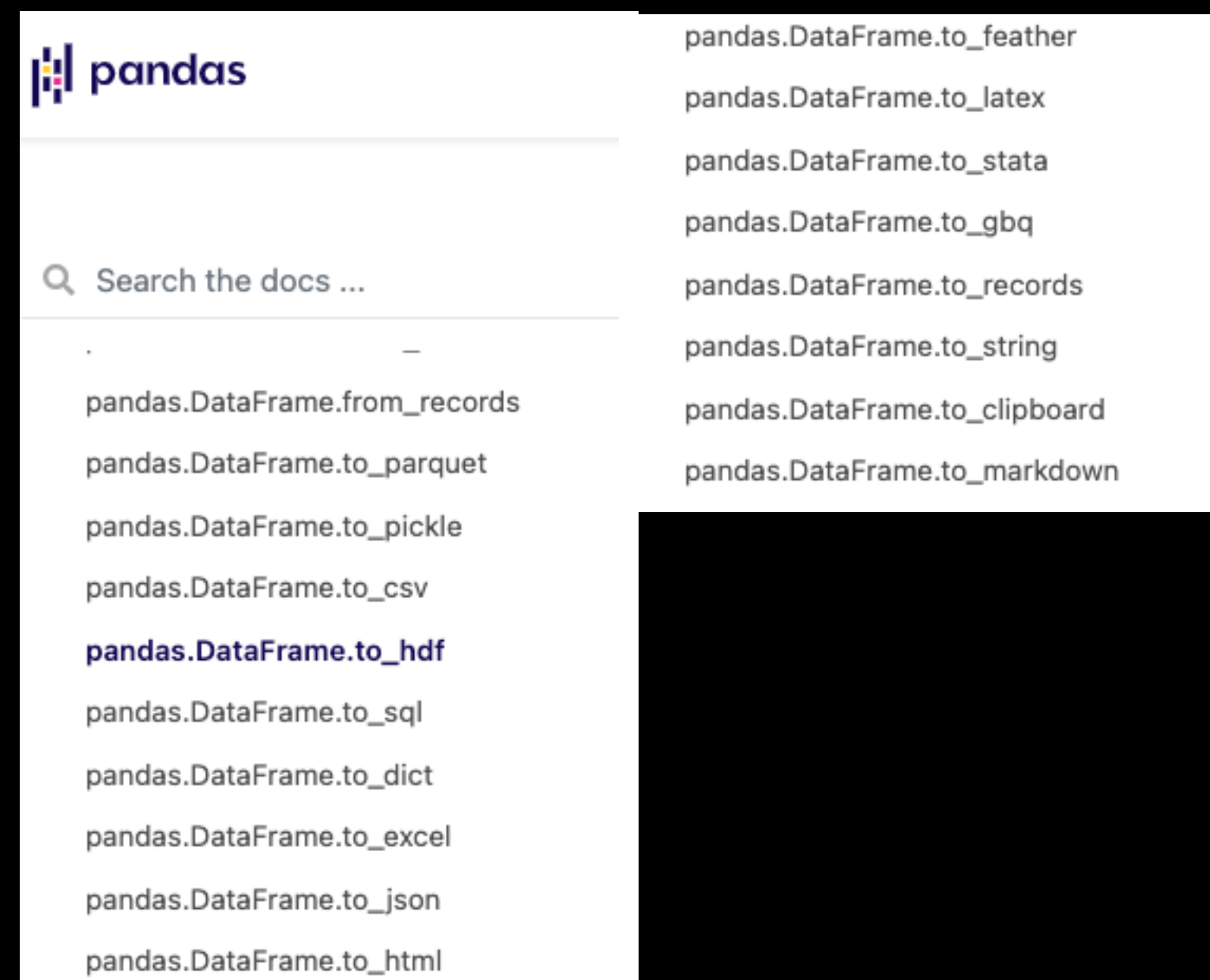
- djd

Exports

- Variables are defined in the scope of your program - whether that is MATLAB, a jupyter notebook, a python IDE, a python script...
- If you want them after the scope of your program doesn't exist anymore, you better save them to disk.

Exports

- `_pickle` is good for saving dictionaries. but you can also pickle anything
 - Some 2 to 3 compatibility issues; also not secure
- `numpy.save`
- Pandas: `df.to_xyz`



Imports v. Exports

- Import and export as used here are not really opposites or paired - import is about bringing in **code**, where export is referring to sending out **data**.
- They are paired here pedagogically by the concept of the scope and namespace of your analysis/coding, but that is it.



Start up a jupyter notebook

Plotting

***the* package**

- In python, originally trying to catch up with matlab: matplotlib

<https://matplotlib.org/stable/gallery/index.html>



Plotting packages

- Seaborn + Pandas



- ipywidgets, Bokeh, plotly/Dash (Jupyter-dash)

- specialized: Brainrender, geoplot,...

